

TEQBALL

Surrealistic IA Edition

Document de Conception Technique

Choix techniques, difficultés rencontrées, architecture, algorithmes

Mamadou Ougailou

Theolien Bierhoff

Steveson Jules

Cours Web3D — Mai 2026

Stack : BabylonJS 6.32 · Havok Physics V2 · TypeScript 5 · Vite

Dépôt : github.com/Mamadou-ougailou/teqball-game

Table des matières

1	Introduction et contexte	3
2	Stack technique et justification des choix	3
2.1	Pourquoi BabylonJS et pas Three.js	3
2.2	Vite comme bundler	3
2.3	Audio : Howler.js et Web Audio API	3
3	Architecture logicielle	4
3.1	Découpage en couches	4
3.2	EventBus : découplage inter-systèmes	4
3.3	RuleEngine : fonctions pures	4
4	Table incurvée procédurale	4
4.1	Génération du maillage par CreateRibbon	5
4.2	Choix de résolution : 8 rails, 32 points	5
4.3	Séparer le visuel du collider	5
5	Moteur physique Havok et stabilisation	6
5.1	Le problème du tunneling	6
5.2	Itérations du solver : accès aux APIs non documentées	6
5.3	Garde anti-tunnel logicielle	6
5.4	Garde anti-oscillation	7
6	Intelligence Artificielle : agent prédictif	7
6.1	Prédiction balistique	8
6.2	Grille spatiale pour la classification de balle	8
6.3	Machine à états de rallye	8
6.4	Courbes de Bézier quadratiques en mode arcade	9
6.5	Équilibrage	9
7	Système d'animations Mixamo	9
7.1	Le problème des armatures multiples	9
7.2	Compensations d'orientation par clip	10
7.3	Synchronisation animation et physique	10
8	Audio procédural	10
8.1	Synthèse à la volée	10
9	Gestion des entrées	11
10	Caméra	11
10.1	Fly-in d'introduction	11
10.2	Suivi dynamique	11
11	Difficultés rencontrées	11
11.1	La table incurvée et les micro-rebonds	11
11.2	Les animations Mixamo	12
11.3	L'équilibrage de l'IA	12

11.4 La machine à états du service	12
12 Ce qui nous a satisfaits	12
13 Limitations et perspectives	13
14 Conclusion	13

Introduction et contexte

Le Teqball est un sport créé en Hongrie en 2012, qui se joue sur une table en forme de double arc de cercle inversé. La balle, un ballon de football standard, doit passer au-dessus d'un filet et rebondir sur la partie convexe du camp adverse. Les joueurs peuvent utiliser toutes les parties du corps sauf les mains, et chaque joueur a droit à trois touches consécutives maximum avant de renvoyer.

Ce qui nous a attirés sur ce sujet, c'est précisément la courbure de la table. Ce n'est pas un simple ping-pong en 3D : la surface parabolique dévie la balle de manière non intuitive, les rebonds ne sont pas plats, et modéliser ça correctement avec un moteur physique web représentait un vrai défi technique.

Le thème "IA Edition" nous a conduits à placer l'humain (Joueur 1) face à un agent prédictif (Joueur 2) qui ne triche pas : il observe le vecteur de vitesse de la balle, résout la cinématique balistique, et prédit l'impact. Nous avons réparti le travail ainsi : Mamadou s'est concentré sur la physique, le moteur de rendu et l'IA ; Theolien a pris en charge les animations Mixamo, le pipeline de modèles et le système de superpouvoirs ; Steveson s'est occupé de l'interface utilisateur, du système audio et de la gestion du score.

Stack technique et justification des choix

Pourquoi BabylonJS et pas Three.js

On a commencé par Three.js, pendant environ deux heures. Le problème s'est posé immédiatement quand on a voulu intégrer Havok : Three.js n'a pas de binding natif, il faut passer par une couche intermédiaire qui convertit les coordonnées, et le debugger physique est quasi inexistant. BabylonJS intègre Havok nativement et partage le même repère de coordonnées, ce qui évite cette friction.

La deuxième raison est le système `AnimationGroup` de BabylonJS. Theolien a exporté 18 clips Mixamo depuis Blender ; les rebrancher sur un squelette commun était une opération que BabylonJS documente précisément. Sous Three.js, on n'a pas trouvé d'équivalent direct pour ce cas précis.

TypeScript strict a été imposé dès le départ. Le projet compte plus de 60 fichiers `.ts` ; les interfaces (`ICharacter`, `IMatchManager`, etc.) ont sauvé plusieurs refactorisations importantes au fil des semaines.

Vite comme bundler

Vite compile le TypeScript en moins de 2 secondes en développement via esbuild. Pour la production, Rollup+Terser produit un bundle déployable directement sur Vercel. Un `Dockerfile` est inclus pour un déploiement alternatif.

Audio : Howler.js et Web Audio API

La musique d'ambiance est gérée par Howler.js pour sa compatibilité cross-browser. Les effets sonores en revanche sont **entièrement procéduraux** : ils sont synthétisés à la volée

via la Web Audio API, sans aucun fichier son chargé depuis le serveur. Le détail technique est en section 8.

Architecture logicielle

Découpage en couches

L'arborescence `src/` suit une séparation en couches que nous avons maintenue strictement tout au long du projet :

Répertoire	Responsabilité
<code>core/</code>	Moteur BabylonJS, EventBus, SceneBuilder, interfaces
<code>entities/</code>	Character, Ball, CurvedTable, Arena (état + maillage)
<code>systems/</code>	BallPhysics, InputManager, CameraManager, CollisionDetector
<code>gameplay/</code>	MatchManager, RuleEngine, RallyManager, ServeManager, IA
<code>animation/</code>	AnimationSystem, retargeting, timing des animations
<code>ui/</code>	HUD, menus, écrans, annonces de points
<code>vfx/</code>	Particules, shaders, traîne de balle
<code>audio/</code>	AudioSystem, MusicManager

Cette séparation n'était pas dans le plan initial : au début, presque tout était dans `main.ts`. On a refactorisé progressivement en identifiant les endroits où un bug dans une couche cassait silencieusement une autre.

EventBus : découplage inter-systèmes

Tous les systèmes communiquent via un EventBus pub/sub minimaliste. Quand la physique détecte un rebond, elle émet `ball:bounce` ; l'AudioSystem s'y abonne et déclenche le son sans avoir de référence directe à la physique. Idem pour le score (`match:pointScored`) et les superpouvoirs (`superpower:activated`).

En pratique, le débogage d'un flux d'événements est moins lisible qu'une pile d'appels directe. On a compensé avec un préfixage strict des namespaces (`ball:`, `match:`, `superpower:`) et des types TypeScript sur chaque payload.

RuleEngine : fonctions pures

Le module `gameplay/RuleEngine.ts` ne contient que des fonctions pures sans dépendance BabylonJS. `evaluatePlayerTouch()`, `evaluateBoundary()`, `recordPoint()` peuvent être testées sous Vitest sans monter de scène 3D. Ce choix était délibéré : les règles du Teqball sont précises (3 touches maximum par joueur, victoire à 25 points avec 2 points d'écart, rotation du service tous les 4 points), et une erreur de comptage de score est catastrophique pour le gameplay.

Table incurvée procédurale

La table de Teqball officielle a une courbure parabolique de 9 cm entre le centre et les bords sur l'axe longitudinal. Reproduire cette géométrie physiquement, pas seulement visuellement, était le premier problème sérieux du projet.

Génération du maillage par CreateRibbon

On a utilisé `MeshBuilder.CreateRibbon` de BabylonJS : on fournit un tableau de rails (tableaux de `Vector3`) et Ribbon construit la surface en reliant les rails par des quads. Cela donne un contrôle précis sur la forme sans fichier 3D externe.

Le profil parabolique suit l'équation :

$$y(z) = y_{\text{surface}} - \Delta y \cdot \left(\frac{z}{L/2} \right)^2 \quad (1)$$

avec $\Delta y = 0.09$ m (courbure réglementaire) et $L/2 = 1.50$ m (demi-longueur).

Listing 1 – Génération des rails paraboliques — `src/entities/CurvedTable.ts`

```
1 const SEGS_X = 8;    // 8 rails en largeur
2 const SEGS_Z = 32;   // 32 points par rail -> courbe visuelle fluide
3
4 for (let i = 0; i <= SEGS_X; i++) {
5   const x = halfWidth * (2 * i / SEGS_X - 1);
6   const rail: Vector3[] = [];
7   for (let j = 0; j <= SEGS_Z; j++) {
8     const z = halfLength * (2 * j / SEGS_Z - 1);
9     const y = surfaceY - curveDrop * (z / halfLength) ** 2;
10    rail.push(new Vector3(x, y, z));
11  }
12  paths.push(rail);
13 }
14 const surface = MeshBuilder.CreateRibbon('teqboard_top', {
15   pathArray: paths,
16   sideOrientation: Mesh.DOUBLESIDE,
17   updatable: false,
18 }, scene);
```

Choix de résolution : 8 rails, 32 points

On a cherché l'équilibre entre fidélité physique et performance. Avec 4 rails, le collider Havok interpolait trop grossièrement entre les quads et la balle sautait à chaque arête. À 16 rails, le gain était imperceptible mais le coût CPU de construction du `PhysicsAggregate` doublait. 8 rails de 32 points, soit 256 vertices pour la surface de jeu, s'est révélé suffisant pour que Havok produise une réponse de contact continue.

Séparer le visuel du collider

Le piédestal (la structure sous la table) est un `CreateBox` purement visuel, intentionnellement exclu des objets transmis à `CourtSetup` pour recevoir un `PhysicsAggregate`. Si on avait oublié cette distinction, la balle aurait eu une collision parasite sur le dessus du piédestal, invisible mais physiquement active. Ça nous est arrivé pendant une demi-journée avant de comprendre d'où venaient les rebonds inexplicables.

Moteur physique Havok et stabilisation

Le problème du tunneling

Avec une balle de rayon 0.11 m se déplaçant à 12–15 m/s, le pas de temps par défaut de Havok (1/60 s) laisse une fenêtre de tunneling de 0.20 à 0.25 m par frame, soit deux fois le rayon de la balle. La balle traversait régulièrement la table sur les frappes puissantes.

Solution retenue : Havok tourne à **120 Hz** via `havokPlugin.setTimeStep(1/120)`, ce qui ramène la fenêtre à environ 0.10 m. Insuffisant seul, donc on a aussi ajouté une garde logicielle (section 5.3).

Itérations du solver : accès aux APIs non documentées

Havok expose des réglages de bas niveau que BabylonJS n’encapsule pas. Pour y accéder, on a dû caster le plugin et lire ses champs privés `_hknv` et `world` :

Listing 2 – Accès aux APIs bas niveau Havok — `src/core/SceneBuilder.ts`

```
1 // Augmentation a 10 iterations vitesse + 4 position
2 // (default Havok : 4 + 2)
3 const hk = (havokPlugin as unknown as {
4   _hknv?: Record<string, (...a: unknown[]) => unknown>
5 })._hknv;
6 const havokWorld = (havokPlugin as unknown as { world?: unknown }).
   world;
7 if (hk && havokWorld !== undefined) {
8   (hk['HP_World_SetNumConstraintSolverVelocityIterations']
9     as (...args: unknown[]) => void)?.(havokWorld, 10);
10  (hk['HP_World_SetNumConstraintSolverPositionIterations']
11    as (...args: unknown[]) => void)?.(havokWorld, 4);
12 }
```

Les 10 itérations de vitesse améliorent la stabilité des contacts glissants sur la surface incurvée. Sur notre machine de test, le coût CPU supplémentaire est d’environ 15%, ce qui reste dans les marges pour cibler 60 FPS.

Garde anti-tunnel logicielle

Même à 120 Hz, certaines frappes rapides tunnelaient encore. On a implémenté une garde dans `BallPhysics.applyAntiTunnelBounce()` basée sur un sweep du segment de trajectoire :

1. Vérifier si la balle a croisé le plan $y = y_{\text{contact}}$ entre le frame précédent et le frame courant.
2. Tester si ce croisement s’est produit au-dessus de la table, via un test AABB en XZ.
3. Si oui, repositionner la balle à $y_{\text{contact}} + \varepsilon$ et inverser v_y avec le coefficient de restitution.

Le test de segment contre AABB utilise l’algorithme des slabs (calcul des $t_{\min}$ et $t_{\max}$ axe par axe), ce qui évite les divisions par zéro quand le mouvement est quasi-parallèle à un axe :

Listing 3 – Test intersection segment–AABB — `BallPhysics.ts`

```

1  const segmentCrossesTableXZ = (from: Vector3, to: Vector3): boolean =>
    {
2    let tMin = 0, tMax = 1;
3    const dx = to.x - from.x;
4    if (Math.abs(dx) < 1e-6) {
5      if (from.x < minX || from.x > maxX) return false;
6    } else {
7      const tx1 = (minX - from.x) / dx;
8      const tx2 = (maxX - from.x) / dx;
9      tMin = Math.max(tMin, Math.min(tx1, tx2));
10     tMax = Math.min(tMax, Math.max(tx1, tx2));
11     if (tMin > tMax) return false;
12   }
13   // idem pour Z...
14   return tMax >= 0 && tMin <= 1;
15 };

```

Garde anti-oscillation

Un autre bug est apparu sur les frappes légères : la balle entrait dans une oscillation haute fréquence contre un joueur ou un bord de table. Havok et le maillage de surface se disputaient la position de la balle à chaque frame.

On a implémenté une fenêtre glissante de 160 ms qui compte les inversions de signe de vitesse en X et Z. Quand le seuil est atteint, la composante oscillante est amortie et la balle est déplacée de 0.06 m dans la direction opposée au joueur le plus proche :

Listing 4 – Détection d’oscillation — `BallPhysics.runOscillationGuard()`

```

1  const flipX =
2    this.oscillationPrevVelocity.x * oscillationVelocity.x < -0.01 &&
3    Math.abs(this.oscillationPrevVelocity.x) >= this.
      ballOscillationMinFlipSpeed &&
4    stepTravel <= this.ballOscillationMaxTravelPerFrame;
5  // ... idem flipZ
6  if (oscillatingX || oscillatingZ) {
7    const dampedVelocity = new Vector3(
8      oscillatingX ? v.x * dampFactor : v.x,
9      Math.max(v.y, popY),
10     oscillatingZ ? v.z * dampFactor : v.z,
11   );
12   physicsBody.setLinearVelocity(dampedVelocity);
13   ball.mesh.position.addInPlace(awayNormal.scale(nudge));
14 }

```

Les constantes (`ballOscillationWindow = 0.16`, `ballOscillationMinFlipSpeed = 1.45 * SCALE`, `ballOscillationMaxTravelPerFrame = 0.16 * SCALE`) ont été calibrées sur une centaine de frappes enregistrées à la main.

Intelligence Artificielle : agent prédictif

L’IA est entièrement dans `src/ai/AIController.ts`. Elle ne lit pas les entrées du clavier ni la direction de frappe du joueur humain. Elle observe uniquement ce que la physique expose : la position et le vecteur de vitesse de la balle, informations disponibles à chaque frame.

Prédiction balistique

Dès que la balle est frappée par le Joueur 1, l'IA résout l'équation du mouvement balistique pour prédire l'heure et le lieu d'impact sur la table :

$$y(t) = y_0 + v_{y0} \cdot t - \frac{1}{2}g \cdot t^2 \quad (2)$$

On cherche t tel que $y(t) = y_{\text{table}}$, ce qui donne un trinôme en t . On retient la racine positive pour laquelle $v_y(t) < 0$ (trajectoire descendante) :

Listing 5 – Prédiction d'impact — `AIController.buildReceptionForecastFromLaunch()`

```
1 const a = -0.5 * this.cfg.gravityAbs;
2 const b = launchVelocity.y;
3 const c = launchPos.y - this.tableTargetY;
4 const disc = b * b - 4 * a * c;
5 if (disc < 0) return; // trajectoire n'atteint pas la table
6
7 const sqrtDisc = Math.sqrt(disc);
8 const t1 = (-b - sqrtDisc) / (2 * a);
9 const t2 = (-b + sqrtDisc) / (2 * a);
10 const candidates = [t1, t2].filter(t => t > 1e-4).sort((x, y) => x - y);
11 let tImpact = -1;
12 for (const t of candidates) {
13   if (launchVelocity.y - this.cfg.gravityAbs * t < 0) {
14     tImpact = t; break;
15   }
16 }
```

Après l'impact sur la table, l'IA recalcule le rebond (avec `TABLE_BOUNCE_RESTITUTION = 0.84`) pour trouver l'apex du second arc, puis choisit le type de frappe selon la hauteur de cet apex.

Grille spatiale pour la classification de balle

L'IA classe chaque position de balle dans une grille $5 \times 4 \times 3$ (5 couloirs en X, 4 bandes de hauteur, 3 profondeurs en Z). Les bandes de hauteur sont les suivantes :

Bande	Fourchette
low	$y < 0.95 \cdot \text{SCALE}$
mid	$0.95 \leq y < 1.55$
high	$1.55 \leq y < 2.35$
veryHigh	$y \geq 2.35$

La combinaison (phase, bande, couloir) est mappée à un type de frappe concret : tête (`kickHead`), poitrine (`kickChest`), genou, ciseau (`kickBicycleLeft`), etc.

Machine à états de rallye

L'IA gère 4 phases par joueur, mises à jour à chaque frame :

— **defense** : balle côté adverse, se repositionner au centre.

- **reception** : balle qui arrive, se placer pour l'impact prédit.
- **preparation** : première ou deuxième touche, mise en position idéale.
- **kick** : troisième touche, frappe offensive vers la cellule cible optimale.

Le choix de la cellule cible (`chooseBestTableCell`) est un score multicritères : distance de l'adversaire à la cellule (maximiser), distance de l'attaquant à la cellule (minimiser), bonus de profondeur pour les frappes hautes.

Courbes de Bézier quadratiques en mode arcade

En mode arcade, la trajectoire de la balle suit une courbe de Bézier quadratique entre l'attaquant et la table, puis entre la table et le défenseur. La formule de De Casteljau :

$$B(t) = (1 - t)^2 P_0 + 2(1 - t)t P_1 + t^2 P_2, \quad t \in [0, 1] \quad (3)$$

La vitesse instantanée est calculée par dérivation et injectée dans le `physicsBody` de la balle, ce qui permet à Havok de maintenir une physique cohérente même sur une trajectoire scriptée.

Équilibrage

La première version de l'IA se positionnait à la précision du millimètre. Contre un humain, c'était inhumain à jouer. On a introduit trois mécanismes de limitation délibérée :

- **Délai de réaction** : le TTL de chaque décision varie de 0.18 à 1.0 s selon la phase et la hauteur de balle.
- **Tolérance de portée** : l'IA ne frappe que si `horizontalDist <= startRange + 0.35 * SCALE`, ce qui lui fait rater les balles légèrement hors de sa zone.
- **Erreur de puissance** : `powerMult` oscille entre 0.82 et 1.28 selon la phase et la distance, ce qui produit des retours parfois trop longs ou trop courts.

Avec ces limitations, l'IA perd environ 40% des points en session normale.

Système d'animations Mixamo

Le problème des armatures multiples

C'est Theolien qui a géré cette partie, et il a passé deux jours dessus. Le modèle joueur est un fichier GLB exporté depuis Blender avec 18 clips Mixamo distincts. Le problème : chaque clip Mixamo embarque son propre squelette indépendant dans le fichier. Résultat, les animations ne s'appliquent qu'au squelette "propriétaire" du clip, et le mesh reste immobile pour les autres.

La solution dans `AnimationSystem.retargetToSkeleton()` consiste à parcourir tous les `AnimationGroup` chargés et à rebrancher leurs `targetedAnimations` sur `skeleton[0]`, le squelette canonique du mesh visible. C'est un remap de bones par nom (`Hips`, `Spine`, `LeftArm`, etc.) qui fonctionne parce que Mixamo utilise une convention de nommage universelle sur tous ses exports.

Compensations d'orientation par clip

Mixamo exporte les personnages avec des orientations variables selon le type d'animation : "idle" face caméra, "jogForward" dos caméra, ciseau-kick avec un décalage d'environ -30° . Corriger ça dans Blender aurait pris du temps que Theolien n'avait plus en fin de projet, donc on a maintenu une table de compensation en degrés dans `Character.ts` :

Listing 6 – Compensations de facing par clip — `Character.ts`

```
1 private static readonly FACING_COMPENSATION_DEG: Array<[string, number  
    ]> = [  
2   ['idle', 5],  
3   ['jogforward', 180],  
4   ['quickjogforward', 180],  
5   ['jogbackward', 180],  
6   ['jogstrafeleft', 90],  
7   ['jogstraferight', 0],  
8   ['headkick', 180],  
9   ['chest_kick', 180],  
10  ['bicycle', 180],  
11  // ...  
12 ];
```

Cette compensation est appliquée pendant toute la durée du clip, puis retirée via un retour interpolé sur 8 frames à la fin.

Synchronisation animation et physique

Chaque frappe a une fenêtre de contact définie dans `data/animationConfig.ts` via `contactFrame` et `contactWindow`. L'impact balle/joueur est déclenché physiquement quand le timer de frappe atteint le ratio `contactFrame / clipLengthFrames`. Sans cette synchronisation, les frappes semblaient flottantes : la jambe touchait la balle visuellement, mais rien ne se passait en physique.

Audio procédural

Synthèse à la volée

Steveson s'est chargé de l'audio. Charger des fichiers WAV ou MP3 pour chaque effet sonore aurait alourdi le déploiement, donc on a opté pour la synthèse procédurale : chaque son est construit à la volée par un graphe d'objets Web Audio.

Le son de rebond sur la table est une oscillation sinusoïdale dont la fréquence descend exponentiellement de 220 Hz à 80 Hz en 80 ms :

Listing 7 – SFX rebond procédural — `AudioSystem.ts`

```
1 private _sfxBounce(ctx: AudioContext): void {  
2   const osc = ctx.createOscillator();  
3   const g = this._gain(ctx, 0.18);  
4   osc.type = 'sine';  
5   osc.frequency.setValueAtTime(220, ctx.currentTime);  
6   osc.frequency.exponentialRampToValueAtTime(80, ctx.currentTime +  
    0.06);  
7   g.gain.exponentialRampToValueAtTime(0.001, ctx.currentTime + 0.08);
```

```
8   osc.connect(g);
9   osc.start();
10  osc.stop(ctx.currentTime + 0.08);
11 }
```

La fanfare de victoire est un arpège en tierce (do-mi-sol, ré-fa#-la, mi-sol#-si) joués en séquence avec un crossfade de 300 ms entre chaque accord.

Gestion des entrées

L'InputManager (`src/systems/InputManager.ts`) gère les bindings AZERTY et QWERTY. La Web API renvoie la valeur de la touche physique via `KeyboardEvent.key`, donc les deux claviers produisent les mêmes codes pour les déplacements principaux.

Un mécanisme "consume-once" est géré par deux tableaux `_kickConsumed[2]` et `_serveConsumed[2]`. Quand `kickPressed()` est appelé, il retourne `true` une seule fois par pression de touche, ce qui évite la répétition automatique des frappes.

Caméra

Fly-in d'introduction

Au démarrage du match, la caméra part d'une position haute et lointaine (`radius = 24 * SCALE`) et descend en 3 secondes vers la position de jeu. L'interpolation utilise un easing $\sin(t \cdot \pi/2)$ pour une décélération fluide en fin de trajectoire :

Listing 8 – `CameraManager.update()` — animation d'intro

```
1 const eased = Math.sin(progress * Math.PI / 2);
2 this._camera.alpha = (Math.PI * 1.5) + (0 - (Math.PI * 1.5)) * eased;
3 this._camera.beta  = 0.3 + (Math.PI / 3.2 - 0.3) * eased;
4 this._camera.radius = (24 * SCALE) + (targetRadius - (24 * SCALE)) *
    eased;
```

Suivi dynamique

En jeu, la cible (`camera.target`) suit la balle avec un lerp de coefficient 0.05 par frame. Le rayon augmente proportionnellement à la hauteur de la balle : $r = r_{\text{base}} + \max(0, y_{\text{ball}} - 2) \times 1.2$. Quand la balle monte très haut sur un smash, la caméra recule automatiquement pour garder la scène dans le cadre.

Difficultés rencontrées

La table incurvée et les micro-rebonds

C'est la difficulté qui nous a pris le plus de temps. La surface Ribbon génère un maillage triangle par triangle, et Havok en contact avec un maillage complexe produisait des micro-sauts à chaque arête de triangle, même avec 32 segments. On a d'abord tenté de lisser les normales du maillage côté Babylon, sans résultat satisfaisant. La solution finale combine trois couches :

1. Physique à 120 Hz pour réduire les sauts interframe.
2. 10 itérations du solver de vitesse pour forcer Havok vers un contact continu.
3. La garde anti-oscillation logicielle comme filet de sécurité final.

Ce n'est pas la solution la plus propre, mais c'est ce qui fonctionne dans les contraintes d'un moteur physique web.

Les animations Mixamo

Chaque nouveau clip ajouté cassait l'orientation d'au moins une frappe. Comme les compensations sont appliquées globalement pendant le clip et retirées à la fin, une valeur incorrecte dans la table de Theolien produisait un personnage qui se retournait en plein smash. Le débogage s'est fait frame par frame en observant l'orientation du mesh pendant les transitions locomotion/frappe.

Theolien a aussi découvert tardivement que certains clips Mixamo exportent avec une translation racine flottante : le personnage se déplace lentement hors du terrain pendant l'animation. Il a fallu pinner les translations Y dans `AnimationSystem.retargetToSkeleton()` pour les clips concernés.

L'équilibrage de l'IA

La première itération de `buildReceptionForecastFromLaunch()` était trop précise. L'IA arrivait systématiquement avant la balle et renvoyait sans ratés. L'équilibrage a demandé plusieurs sessions de test en binôme (Mamadou jouait, Steveson annotait les patterns de défaite) pour calibrer les TTL variables et les tolérances de portée à un niveau de difficulté jouable.

La machine à états du service

La gestion du service est une machine à états à 4 phases (`ready` → `toss` → `strike` → `flight`). La version initiale dans `main.ts` avec des variables globales est devenue ingérable dès qu'on a ajouté des cas limites : service raté, balle sortie pendant le toss, timeout de vol. Le refactoring vers `ServeManager.ts` a clarifié les responsabilités, mais a introduit un couplage inverse : `ServeManager` dépend de callbacks vers `main.ts` pour les opérations qui nécessitent l'état de closure du fichier principal. C'est une dette technique qu'on n'a pas eu le temps de complètement éliminer.

Ce qui nous a satisfaits

Le sweep test anti-tunnel. Ce n'était pas obligatoire. Havok à 120 Hz couvre 95% des cas. Mais Mamadou a voulu implémenter un vrai sweep test de segment contre une AABB avec extrapolation du point d'impact et restitution correcte de la vitesse normale. C'est 80 lignes de code que l'on comprend entièrement et dont le comportement est prévisible.

L'IA qui ne triche pas. L'IA observe uniquement le vecteur de vitesse de la balle, une information physiquement observable dans le jeu. Si la balle rebondit sur une paroi latérale, l'IA révisé son forecast au frame suivant. Ce comportement émerge de l'architecture sans cas spéciaux.

L'audio entièrement synthétisé. Que la fanfare de victoire, les rebonds et les frappes soient construits en JavaScript pur à partir d'oscillateurs et d'enveloppes, sans aucun fichier audio, est une contrainte que Steveson a transformée en avantage : le déploiement ne dépend d'aucun asset audio externe.

Limitations et perspectives

- **IK (Inverse Kinematics)** : Le fichier `IKController.ts` existe mais est vide (TODO Phase 5). Avec de l'IK, les pieds des personnages s'adaptent correctement à la courbure de la table.
- **Multijoueur réseau** : L'architecture EventBus est compatible avec une couche de synchronisation réseau, mais nous n'avons pas eu le temps de l'implémenter.
- **Mobile** : Le jeu nécessite un clavier. L'ajout de contrôles tactiles se ferait via un InputManager alternatif ; le reste du code n'aurait pas à changer.

Conclusion

Ce projet nous a confrontés à des problèmes qu'on n'avait pas anticipés en commençant. La physique d'une balle rapide sur une surface courbe est beaucoup plus délicate à stabiliser qu'une physique de plateforme classique. Le passage de Havok à 120 Hz, le sweep test, la garde anti-oscillation : aucun de ces composants n'était dans l'architecture initiale. Ils sont apparus de session de débogage en session de débogage, chacun résolvant un problème spécifique qui avait résisté à la solution précédente.

Sur la répartition du travail : les trois parties du projet (physique/IA, animations, audio/UI) ont suivi des chemins très différents. Mamadou a travaillé sur des problèmes mathématiques relativement bien définis. Theolien a passé plus de temps à se battre avec des comportements d'exporteurs 3D que sur des algorithmes. Steveson a eu l'avantage que la synthèse audio Web est bien documentée, mais le moins bon côté du débogage de timing entre les événements EventBus et les animations.

En termes de résultat final, le jeu tourne, les règles du Teqball sont respectées, l'IA constitue un adversaire correct, et aucun fichier audio n'est chargé depuis le serveur.

Dépôt source complet : <https://github.com/Mamadou-ougailou/teqball-game>